

TrafficAI — Signal Prediction API

Project Documentation

1. Overview

TrafficAI is a smart traffic management module that uses computer vision to detect vehicles in a live traffic image and recommend adaptive signal timings (green, yellow, red) based on real-time vehicle density. This document covers the API design, the tools/libraries used, request/response structure, and setup/run instructions.

2. Tech Stack

Backend Framework

- **FastAPI** — Python web framework used to expose the REST API. Chosen for its speed, automatic request validation, and built-in interactive documentation (Swagger UI).
- **Uvicorn** — ASGI server used to run the FastAPI application.

Computer Vision / Machine Learning

- **Ultralytics YOLO (yolo11s.pt)** — Pre-trained object detection model used to detect vehicles (cars, buses, trucks, motorcycles, bicycles) in traffic images.
- **OpenCV (opencv-python-headless)** — Used for image decoding, drawing bounding boxes/labels on detected vehicles, and encoding the final annotated image.
- **NumPy** — Used internally for image array handling.

Data Validation

- **Pydantic** — Used to define and validate the structure of API response data.

Supporting Tools

- **Postman** — Used to manually test the API endpoints (file upload + form data).
 - **python-multipart** — Required by FastAPI to handle multipart/form-data (needed for image file uploads).
-

3. How the System Works (Flow)

1. Client sends a traffic image + signal metadata to the API.
2. The image is decoded and passed through the YOLO model.

3. The model detects vehicles and counts them by class (car, bus, truck, motorcycle, bicycle).
 4. Bounding boxes and labels are drawn on the image (annotated output).
 5. Vehicle count, road width, and previous cycle data are used to calculate:
 - Traffic density (low / medium / high)
 - Recommended green light duration
 - Recommended yellow light duration
 - Recommended red light duration
 - A confidence score for the prediction
 6. The API returns the results as JSON, including the annotated image encoded in Base64.
-

4. API Endpoints

5.1 Health Check

Endpoint: `GET /health`

Purpose: Confirms the API server is running.

Sample Response:

```
{ "status": "ok" }
```

5.2 Signal Prediction

Endpoint: `POST /predict/signal`

Content-Type: `multipart/form-data`

Purpose: Detects vehicles in an uploaded traffic image and returns recommended signal timings.

Input Parameters

Field	Type	Required	Description
traffic_image	File (image)	Yes	The traffic camera image to analyze
road_width	Float	Yes	Width of the road in meters (affects timing calculation)
signal_id	String	Yes	Unique identifier for the traffic signal
timestamp	String	No	Time of capture (auto-filled with current UTC time if omitted)
previous_vehicle_count	Integer	No (default 0)	Vehicle count from the previous signal cycle
previous_red_light_time	Float	No (default 0.0)	Red light duration from the previous cycle

Output Fields

Field	Type	Description
vehicle_count	Integer	Number of vehicles detected in the current image
annotated_image	String (Base64)	JPEG image with bounding boxes drawn around detected vehicles
traffic_density	String	Classified as "low", "medium", or "high"
recommended_green_time	Float	Suggested green light duration (seconds)
recommended_yellow_time	Float	Suggested yellow light duration (seconds)
recommended_red_time	Float	Suggested red light duration (seconds)
confidence_score	Float	Prediction confidence (0.0 to 1.0)
signal_id	String	Echoes the signal ID from the request
timestamp	String	Echoes or auto-generates the timestamp

Sample Response

```
{ "vehicle_count": 8, "annotated_image": "<base64-encoded-jpeg-string>",  
  "traffic_density": "medium", "recommended_green_time": 20.0,  
  "recommended_yellow_time": 3.0, "recommended_red_time": 18.0,
```

"confidence_score": 0.85, "signal_id": "SIG_001", "timestamp": "2025-01-01T10:00:00" }

5. Setup & Run Guide

Step 1: Install Dependencies

Navigate to the project root folder and install all required Python packages listed in requirements.txt using pip.

Step 2: Download Model Weights

Run the model download script once. This fetches the YOLO model weights and stores them inside the models folder so the API can use them for detection.

Step 3: Start the Server

Launch the FastAPI application using Uvicorn, specifying the app file and enabling auto-reload during development. The server will run locally on port 8000 by default.

Step 4: Verify the Server is Running

Open a browser or Postman and send a GET request to the health check endpoint. A successful response confirms the server is active.

Step 5: Test the Prediction Endpoint

Using Postman, send a POST request to the signal prediction endpoint with form-data containing the traffic image file and the required text fields (road width, signal ID, etc.). The response will include the vehicle count, recommended signal timings, and the annotated image in Base64 format.

Step 6: View the Annotated Image (Optional)

Since Postman does not render Base64 strings as images automatically, the annotated_image value can be copied into an online Base64-to-image viewer, or rendered using Postman's built-in Visualize tab with a simple HTML image template.

6. Known Limitations

- The yolo11s.pt model is a general-purpose model trained on the COCO dataset. It detects standard vehicle categories (car, bus, truck, motorcycle, bicycle) but is **not specifically fine-tuned for Indian traffic conditions** — vehicles like autos, e-rickshaws, and non-standard local vehicles may not be detected accurately.
- For improved accuracy on Indian roads, the model would need to be fine-tuned on an Indian traffic dataset (e.g., the Indian Driving Dataset) using a custom training script.

- Signal timing logic currently uses simplified rule-based formulas rather than a trained predictive model; this can be replaced with a machine learning model in future iterations.
-

7. Future Improvements

- Fine-tune YOLO on Indian vehicle dataset for higher detection accuracy.
- Add authentication/API key protection for production use.
- Add database logging of each prediction for historical traffic analysis.
- Integrate with the other two predictors (bus lane violation detection, parking occupancy detection) under a unified API gateway.
- Containerize the application with Docker for easier deployment.